



Note: There is an object called `localStorage`, which can be used to store data in key value pairs. This can be implemented using the function `window.localStorage.setItem("key", value);`

The arguments to pass through these methods are:

- `openDatabase(database_name, database_version, database_displayname, database_size)`: This will create a new database with the details specified, i.e., name, version, etc.
- `database.transaction(insertDB, errorDB, successDB)`: This transaction method will lead to running 'insertDB', where all the executesql statements will be written, and success and error will return messages for success and failure, respectively.
- `changeVersion('old_version', 'new_version')`: This method will change the version from the old to the new.
- `executeSql('SQL_STATEMENTS_TO_BE_WRITTEN_HERE')`: All the SQL statements will be written in the executesql method.

Let's see what these arguments will actually do when put together.

```
// Create a file name Storage.html
<!DOCTYPE html >
<html>
<head>
  <title>My Page</title>
  <script type="text/javascript" charset="utf-8"
src="cordova-2.7.0.js"></script>    (1)
  <script type="text/javascript" charset="utf-8">

    document.addEventListener("deviceready", onDeviceReady,
false);    (2)

    function onDeviceReady()
    {
      var db= window.openDatabase("MyDatabase", "1.0", "My
Database", 1000000);    (3)
      db.transaction(insertDB, errorDB, successDB)    (4)
    }

    function insertDB(tr) {
      tr.executeSql('DROP TABLE IF EXISTS CountryList';
(5)
      tr.executeSql('CREATE TABLE IF NOT EXISTS CountryList
(countrycode unique, name)'); (6)
      tr.executeSql('INSERT INTO CountryList (countrycode,
name) VALUES (61, "Australia")'); (7)
      tr.executeSql('INSERT INTO CountryList (countrycode,
name) VALUES (55, "Brazil")');
      tr.executeSql('INSERT INTO CountryList (countrycode,
name) VALUES (1, "Canada")');
      tr.executeSql('INSERT INTO CountryList (countrycode,
name) VALUES (91, "India")');
```

```
}

function successDB() {
  alert("success!");
  var db = window.openDatabase("MyDatabase", "1.0", "My
Database", 1000000);
  db.transaction(queryDB, errorDB);
}

function queryDB(tx)
{
  tr.executeSql('SELECT * FROM CountryList', [], querySuccess,
errorDB);    (8)
}

function querySuccess(tr, output)
{
  console.log("No of rows returned is " + output.rows.
length);    (9) and (10)

  var len = output.rows.length;

  for (var i=0; i<len; i++)
  {
    console.log("Row = " + i + " ID = " + output.rows.
item(i).code + " Data = " + output.rows.item(i).name);
  }
}

function errorDB(error) {
  alert("Error processing SQL: "+error.code);
}

</script> </head>
<body>
<h2>Fetching Country list from databases.</h2>
<h4> Showing Country list.. </h4>
</body></html>
```

The output of the above code is given in Figure 1.

The code shown above displays the list of countries saved in the database. Here is the explanation:

```
No of rows returned is 4 at file:///android_asset/www/storage.html:
ml:41
Row = 0 ID = 61 Data = Australia
Row = 0 ID = 61 Data = Australia at file:///android_asset/www/s
torage.html:47
Row = 1 ID = 55 Data = Brazil
Row = 1 ID = 55 Data = Brazil at file:///android_asset/www/stor
age.html:47
Row = 2 ID = 1 Data = Canada
Row = 2 ID = 1 Data = Canada at file:///android_asset/www/stora
ge.html:47
Row = 3 ID = 91 Data = India
Row = 3 ID = 91 Data = India at file:///android_asset/www/stora =
```

Figure 1: Storage